

Disciplina: Análise de Algoritmos
Curso: Engenharia da Computação – 8º semestre
Dupla: Luiz Felipe e Stefany
Algoritmo: Shell Sort

Análise do Algoritmo Shell Sort

1. Introdução

O Shell Sort é um algoritmo de ordenação proposto por Donald Shell em 1959, com o objetivo de melhorar o desempenho do Insertion Sort. A principal inovação do método está na comparação de elementos distantes entre si, reduzindo significativamente o número de movimentações necessárias durante a ordenação.

Segundo Thomas H. Cormen et al. (2009), o Shell Sort introduz uma abordagem baseada na ordenação por intervalos, permitindo reduzir a quantidade de deslocamentos necessários durante o processo de ordenação.

2. Funcionamento do Algoritmo

O Shell Sort utiliza uma sequência de intervalos (*gaps*) para dividir o vetor em sublistas menores. Cada sublista é ordenada individualmente, e o intervalo é reduzido progressivamente até atingir o valor 1.

De acordo com Thomas H. Cormen et al. (2009), esse processo permite que elementos distantes sejam reposicionados rapidamente, diminuindo o esforço necessário na fase final da ordenação.

3. Exemplo de Execução do Shell Sort

Para melhor compreensão do funcionamento do algoritmo Shell Sort, considere o vetor inicial:

[8, 5, 3, 7, 6, 2, 1, 4]

O tamanho do vetor é 8, portanto o valor inicial do intervalo (*gap*) será 4.

3.1 Primeira Iteração (gap = 4)

Nesta etapa, são comparados elementos que estão a uma distância de 4 posições entre si. Assim, formam-se os seguintes pares:

- (0,4): 8 e 6
- (1,5): 5 e 2

- (2,6): 3 e 1
- (3,7): 7 e 4

Cada par é ordenado individualmente:

- $8 > 6 \rightarrow \text{troca} \rightarrow [6, 5, 3, 7, 8, 2, 1, 4]$
- $5 > 2 \rightarrow \text{troca} \rightarrow [6, 2, 3, 7, 8, 5, 1, 4]$
- $3 > 1 \rightarrow \text{troca} \rightarrow [6, 2, 1, 7, 8, 5, 3, 4]$
- $7 > 4 \rightarrow \text{troca} \rightarrow [6, 2, 1, 4, 8, 5, 3, 7]$

Resultado ao final da iteração:

[6, 2, 1, 4, 8, 5, 3, 7]

Nesta fase, observa-se que valores maiores começam a se deslocar para o final do vetor, enquanto valores menores aproximam-se do início.

3.2 Segunda Iteração (gap = 2)

O intervalo é reduzido para 2, e o vetor é dividido em dois grupos:

- Índices pares: [6, 1, 8, 3]
- Índices ímpares: [2, 4, 5, 7]

Cada grupo é ordenado separadamente:

Grupo 1:

- $6 > 1 \rightarrow \text{troca} \rightarrow [1, 6, 8, 3]$
- $8 > 3 \rightarrow \text{troca} \rightarrow [1, 6, 3, 8]$
- $6 > 3 \rightarrow \text{troca} \rightarrow [1, 3, 6, 8]$

Grupo 2:

- Já se encontra ordenado

Atualizando o vetor completo:

[1, 2, 3, 4, 6, 5, 8, 7]

Neste ponto, o vetor encontra-se parcialmente ordenado, restando apenas pequenos ajustes.

3.3 Terceira Iteração (gap = 1)

Com o intervalo igual a 1, o algoritmo passa a se comportar como o Insertion Sort.

Realizando as comparações finais:

- $6 > 5 \rightarrow \text{troca} \rightarrow [1, 2, 3, 4, 5, 6, 8, 7]$

- $8 > 7 \rightarrow \text{troca} \rightarrow [1, 2, 3, 4, 5, 6, 7, 8]$

Resultado final:

[1, 2, 3, 4, 5, 6, 7, 8]

3.4 Análise do Exemplo

O exemplo apresentado demonstra que o Shell Sort realiza a ordenação de forma gradual, utilizando diferentes intervalos para reduzir a desordem do vetor. Inicialmente, grandes deslocamentos são realizados, permitindo reposicionar elementos distantes. Em seguida, com a redução do intervalo, o algoritmo realiza ajustes mais refinados.

Dessa forma, quando o intervalo chega a 1, o vetor já está quase ordenado, tornando a etapa final mais eficiente. Esse comportamento explica o melhor desempenho do Shell Sort em comparação com algoritmos simples de ordenação.

4. Análise do Algoritmo com Base na Implementação

Para compreender melhor o funcionamento do Shell Sort, é importante observar sua estrutura em código, pois a análise de desempenho está diretamente relacionada aos seus laços de repetição.

A seguir, apresenta-se uma implementação em linguagem C:

```
void shellSort(int vetor[], int n) {
    int gap, i, j, temp;

    // Define o gap inicial
    for (gap = n / 2; gap > 0; gap /= 2) {

        // Percorre os elementos a partir do gap
        for (i = gap; i < n; i++) {
            temp = vetor[i];

            // Ordenação por inserção com intervalo (gap)
            for (j = i; j >= gap && vetor[j - gap] > temp; j -= gap) {
                vetor[j] = vetor[j - gap];
            }

            vetor[j] = temp;
        }
    }
}
```

4.1 Interpretação do Código

O algoritmo é composto por três estruturas principais:

- Um laço externo que controla o valor do *gap*
- Um laço intermediário que percorre o vetor
- Um laço interno que realiza as comparações e trocas (semelhante ao Insertion Sort)

O comportamento do algoritmo depende diretamente da interação entre esses três níveis.

4.2 Análise de Complexidade

- Laço externo (controle do gap):

```
for (gap = n / 2; gap > 0; gap /= 2)
```

O valor do *gap* é reduzido pela metade a cada iteração

Número de execuções: aproximadamente $\log_2(n)$

- Laço intermediário:

```
for (i = gap; i < n; i++)
```

Percorre praticamente todo o vetor

Complexidade: $O(n)$

- Laço interno:

```
for (j = i; j >= gap && vetor[j - gap] > temp; j -= gap)
```

Responsável pelas trocas

No pior caso, pode percorrer grande parte do vetor

4.3 Complexidade Final

A combinação desses laços resulta nas seguintes complexidades:

- **Melhor caso:** $O(n \log n)$
(quando o vetor já está quase ordenado)
- **Caso médio:** aproximadamente $O(n^{1.5})$
(valor empírico baseado em estudos práticos)
- **Pior caso:** $O(n^2)$
(quando há muitas inversões no vetor)

4.4 Interpretação Teórica

Diferente de algoritmos como o Merge Sort e o Quick Sort, o Shell Sort não possui uma fórmula fechada exata para sua complexidade.

Isso ocorre porque seu desempenho depende diretamente da sequência de *gaps* utilizada. De acordo com Thomas H. Cormen et al. (2009), diferentes escolhas de intervalos podem impactar significativamente o número de comparações e movimentações realizadas pelo algoritmo.

4.5 Análise Intuitiva

A eficiência do algoritmo pode ser compreendida da seguinte forma:

- Com *gaps* grandes → reduz rapidamente a desordem global
- Com *gaps* médios → organiza grupos parcialmente
- Com *gap* = 1 → finaliza com poucos ajustes

Esse comportamento reduz o custo da etapa final, que seria mais cara caso fosse executada diretamente como um Insertion Sort.

4.6 Conclusão da Análise

A análise baseada na implementação mostra que o Shell Sort é um algoritmo que equilibra simplicidade e eficiência. Embora sua complexidade no pior caso seja quadrática, seu desempenho prático tende a ser superior ao de algoritmos simples devido à estratégia de redução progressiva da desordem.

5. Sequências de Gaps

Diversas sequências foram propostas ao longo do tempo:

- Shell: $n/2, n/4, n/8, \dots$
- Hibbard: $2^k - 1$
- Knuth: $(3^k - 1)/2$

A sequência de Knuth é amplamente utilizada por apresentar bons resultados práticos (SEdgeWICK, 2011).

6. Comparação com Outros Algoritmos

Em comparação com algoritmos como o **Quick Sort** e o Merge Sort, o Shell Sort apresenta menor previsibilidade de desempenho.

No entanto, de acordo com Thomas H. Cormen et al. (2009), o Shell Sort pode ser considerado uma alternativa viável em cenários onde simplicidade de implementação e baixo consumo de memória são fatores relevantes.

7. Conclusão

O Shell Sort é um algoritmo relevante na evolução dos métodos de ordenação, introduzindo a ideia de comparações por intervalos. Apesar de suas limitações teóricas, ele

apresenta bom desempenho prático em diversos contextos, especialmente para conjuntos de dados de tamanho médio.

REFERÊNCIAS

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. *Algoritmos: teoria e prática*. 3. ed. Rio de Janeiro: Elsevier, 2012.

SEDGEWICK, Robert; WAYNE, Kevin. *Algoritmos*. 4. ed. São Paulo: Pearson, 2011.